

UmweltRover – Probleme im Code

KRITISCHE PROBLEME

1. Bewegungserkennung ist einseitig

Datei: `motion_detector.py`

```
diff = np.subtract(gray1.astype(np.int16), gray2.astype(np.int16))
diff = np.clip(diff, 0, 255).astype(np.uint8)
```

Nur eine Bewegungsrichtung (hell zu dunkel) wird erkannt. Die entgegengesetzte Richtung erzeugt negative Werte, die auf 0 abgeschnitten werden und dadurch vollständig verloren gehen.

2. Triangulation ohne Kamera-Korrespondenz-Zuordnung

Datei: `main_server_2.py` ruft `triangulation_3.py::trianguliere_motion_pixels()` auf

Die Funktion sammelt alle Sichtlinien aller Kameras in einer gemeinsamen Least-Squares-Lösung und liefert einen einzigen Schnittpunkt – ohne Zuordnung, welcher Blob zu welchem Objekt gehört. Bei mehreren Blobs pro Kamera entsteht ein bedeutungsloser Mischpunkt.

3. Realer Frame-Abstand durch ISP-Latenz dominiert

Datei: `pi_client_final.py`

```
frame1 = picam.capture_array()
time.sleep(CAPTURE_DELAY_S) # 0.020s
frame2 = picam.capture_array()
```

Ein einzelner `capture_array()`-Aufruf bei voller Auflösung dauert 300–600ms. Die vorgesehene 20ms-Pause ist gegenüber dieser Aufnahmedauer vernachlässigbar; der tatsächliche Frame-Abstand liegt weit über dem beabsichtigten Wert.

4. GPS: 2D-Fix und 0.0-Höhe werden als gültiger Fallback akzeptiert

Datei: `gps_reader.py`

```
valid_fix = (mode >= 2) and (lat is not None) and (lon is not None) and (lat
!= 0.0) and (lon != 0.0)
_last_valid_raw = {
    "lat": float(lat), "lon": float(lon),
    "alt": float(alt) if alt is not None else 0.0,
    "timestamp": _iso_now_utc(),
}
```

`mode == 2` ist ein 2D-Fix ohne verifizierte Höhe, wird aber akzeptiert. Ein Höhenwert von 0.0 kann ungefiltert in `_last_valid_raw` landen und als Fallback in `get_gps_data()` verwendet werden.

WICHTIGE PROBLEME

5. GPS-Referenzpunkt wird dauerhaft eingefroren

Datei: `main_server_2.py`, `processing_worker()`

```
if gps_reference_point is None:
    gps_reference_point = (...)
```

Wird einmalig beim ersten Event gesetzt und nie mehr aktualisiert. Ein fehlerhafter erster Wert (siehe Punkt 4) verschiebt alle nachfolgenden Berechnungen der Session.

6. `min_area=1` erzeugt Rauschen als eigene Blobs

Datei: `main_server_2.py`

```
motion_pixels = md.detect_motion_simple(..., min_area=1, ...)
```

Lässt praktisch jede zusammenhängende Pixelregion als Blob durch, inklusive Rauschpixel. Verschärft Punkt 2.

7. Sensordaten werden nach den Kamera-Frames erfasst, nicht davor

Datei: `pi_client_final.py`

```
frame1 = picam.capture_array()
time.sleep(CAPTURE_DELAY_S)
frame2 = picam.capture_array()
...
gps_selected, gps_mode = get_selected_gps()
ori_selected, imu_mode = get_selected_orientation()
```

GPS, IMU, Temperatur und Lichtstärke werden erst nach beiden Aufnahmen gelesen. Die gespeicherten Sensorwerte beschreiben dadurch einen anderen Zeitpunkt als die Bilder.

8. I2C-Fehlerbehandlung der IMU setzt den Bus nicht zurück

Datei: `orientation_reader.py`

```
if _error_count >= ERROR_THRESHOLD:
    print("[ORI] Too many I2C errors, resetting I2C bus...")
    time.sleep(0.5)
    _error_count = 0
```

Das `_i2c` -Handle wird trotz der Meldung nie geschlossen oder neu erstellt.

9. Keine Überprüfung der Zeitsynchronisation vor dem Auslösen

Datei: Server + Pi-Clients

Keine Laufzeitprüfung, ob alle drei Pis und der Mainserver tatsächlich per NTP synchron sind. Eine Abweichung von mehr als ~50ms verletzt die Gleichzeitigkeits-Annahme der Triangulation, ohne sichtbar zu werden.

NIEDRIGE PRIORITÄT

10. Dash-Trigger-UI ignoriert Änderungen am „lead time“-Feld

Datei: `main_server_2.py`, Callback `trigger_start_stop()`

```
lead_s = float(lead_val) if lead_val is not None else TRIGGER_DEFAULT_LEAD_S
lead_s = max(float(TRIGGER_MIN_LEAD_S), float(trigger_lead_s)) # nutzt
alten Wert statt lead_s
```

Das Eingabefeld „lead time (s)“ im Dashboard hat dadurch keine Wirkung.

11. Barometrische Höhe nutzt festen Referenzdruck

Datei: `BME280.py`

```
def get_altitude_baro(sea_level_pressure_hpa=1013.25):
```

Wird überall ohne Argument aufgerufen, verhält sich faktisch wie hartcodiert.

12. POSITION -Variable im Pi-Client ist toter Code

Datei: `pi_client_final.py`

```
POSITION = {"x": 0.0, "y": 0.0, "z": 1.2}
```

Wird gesendet, serverseitig aber nie gelesen.

13. Tote Funktionen referenzieren nicht existierende globale Variablen

Datei: `orientation_web.py`

`get_visible_altitude()`, `get_altitude_mode()`, `get_manual_altitude()` referenzieren `ALTITUDE_MODE / MANUAL_ALTITUDE`, die nirgends definiert sind. Aktuell unerreichbar (nicht importiert), würde bei Aufruf sofort `NameError` auslösen.

HARDWARE-HINWEIS (kein Code-Bug)

H1. NEO-6M GPS hat inhärente $\pm 5\text{--}10\text{m}$ Höhenungenauigkeit

Datei: Hardware

Consumer-GPS-Modul mit typisch $\pm 2\text{--}3\text{m}$ horizontaler und $\pm 5\text{--}10\text{m}$ vertikaler Genauigkeit – durch Code allein nicht vollständig kompensierbar.